

Database Course

Security and User Management for Protecting Data

Lotfi NAJDI

Polydisciplinary Faculty of Taroudant

2025-2026

What is Database Security?

Database security is about controlling **who can access what** in your database

Think of it as a **“multi-level security system”** for a building 

Different people need different levels of access to different areas

A Simple Analogy

Think of database security like a **hotel access system**.

- **Guest key card**: Basic room access (regular user)
- **Staff key card**: Multiple rooms + service areas (application role)
- **Manager key card**: All rooms + offices (admin role)
- **Master key**: Every door in the building (superuser)

Each level of access serves a specific purpose

PostgreSQL Security Architecture

PostgreSQL uses a **layered security model**:

1. **Authentication** - Who are you? (Identity verification)
2. **Authorization** - What can you do? (Permission checking)
3. **Access Control** - Where can you go? (Object-level security)
4. **Row-Level Security** - Which data can you see? (Fine-grained control)

A Database

- In PostgreSQL, a database is a structured collection of data stored in a way that allows for efficient access and management.
- Each database is isolated from others, ensuring data integrity and security.
- For example, in a multi-tenant application, each client might have their own database to keep data separate.

Database Creation

```
1 -- Create a new database  
2 CREATE DATABASE company_db;
```

Schema

- In PostgreSQL, a schema is a logical container for database objects (tables, views, functions, etc.).
- A schema is a namespace or a “folder” inside a database that contains objects.
- It’s the primary way to organize, manage and secure your database objects.

Schema

- Think of a digital department store.
- Each schema is a different business unit with its own data and access rules.
- For example, a company might have separate schemas for different departments (sales, finance, hr) within the same database.

Schema creation

```
1 -- Create a new schema  
2 CREATE SCHEMA sales;
```

The Public Schema

- By default, PostgreSQL creates a schema named “public” in every new database.
- All users have access to the public schema unless explicitly restricted.
- It’s a convenient place to store shared objects but can be a security risk if not managed properly.

Schemas in Action - E-commerce Platform

In an e-commerce application, schemas act as distinct business units, each with its own data and security rules.

- **sales** - The core of the business : * **customers, orders, payments, shipments**
- **inventory** - Manages all the products being sold: * **products, suppliers, warehouses, product_categories**
- **hr** - Holds sensitive employee information: * **employees, departments, job_positions, salaries**
- **analytics** - Stores aggregated data for business intelligence: * **sales_summary_daily, customer_behavior_logs, product_performance_reports**

Schemas in Action: University Management System



A university database manages distinct, sensitive areas of operation.

- **admissions** - Handles the entire application process: * **applicants**, **applications**, **interview_schedules**, **recommendation_letters**
- **academics** - Manages student learning and official records: * **students**, **courses**, **professors**, **enrollments**, **grades**
- **hr** - Manages all faculty and staff employment details: * **faculty**, **staff**, **payroll**, **contracts**
- **library** - Tracks all library assets and activities: * **books**, **authors**, **loans**, **book_reservations**

Key Benefit: The admissions office can be granted full access to the **admissions** schema while being completely blocked from viewing sensitive

Custom Schemas

```
1 -- Example: Creating a table in the sales schema
2 CREATE TABLE sales.customers (
3     customer_id SERIAL PRIMARY KEY,
4     name VARCHAR(100),
5     email VARCHAR(255)
6 );
```

```
1 -- Example: Accessing a table in the sales schema
2 SELECT * FROM sales.customers;
```

Users

- A user is an account that can connect to the database and perform actions based on assigned privileges.
- Users are created to manage access and ensure security.

User Creation

```
1 -- Create a new user  
2 CREATE USER app_user WITH PASSWORD 'secure_password';
```

User Creation Attributes

- CREATEDB: Allows the user to create new databases.

```
1 CREATE USER app_user CREATEDB;
```

- CREATEROLE: Allows the user to create, alter, and drop other roles.

```
1 CREATE USER app_user CREATEROLE;
```

- SUPERUSER: Grants all privileges, including bypassing all permission checks (use with caution).

```
1 CREATE USER app_user SUPERUSER;
```

- VALID UNTIL: Sets an expiration date for the user account.

```
1 ALTER USER app_user VALID UNTIL 'tomorrow';
```


Privileges

- Users can be granted various privileges such as CONNECT, SELECT, INSERT, UPDATE, DELETE, etc.
- Privileges determine what actions a user can perform on database objects.
- For example, a read-only user might only have SELECT privileges on certain tables.

Granting and Revoking Privileges to Users

- To grant privileges to a user, you can use the GRANT command.

```
1 GRANT SELECT ON sales.customers TO app_user;
```

- To revoke privileges from a user, you can use the REVOKE command.

```
1 REVOKE SELECT ON sales.customers FROM app_user;
```

User, Database and Schema Relationship

- A user connects to a specific database.
- Within that database, the user can access objects in various schemas based on their privileges.
- For example, a user might connect to the “company_db” database and have access to the “sales” schema but not the “hr” schema.
- This separation helps enforce security and organization within the database.

What About Roles?

- A role is a more general concept that can represent either a user or a group of users.
- Roles can own database objects and have privileges assigned to them.
- Roles can be granted to other roles, allowing for hierarchical permission structures.

Users vs Roles: The Difference

- In PostgreSQL a role can represent either a single user or a group of privileges.
- With LOGIN attribute a role can act as a user. Users are just roles with LOGIN privilege!

```
1 CREATE ROLE app_user WITH LOGIN PASSWORD 'secure_password';  
2 -- app_user can now log in like a user
```

- Without it, the role cannot log in. Role could be used to group privileges.

```
1 CREATE ROLE manager;  
2 -- manager cannot log in, used for grouping privileges
```

View Existing Roles

```
1 SELECT rolname, rolsuper, rolcreatedb, rolcanlogin  
2 FROM pg_roles  
3 ORDER BY rolname;
```

Granting Privileges

- Roles can be granted privileges on database objects (tables, schemas, databases).
- Users can be granted roles to inherit their privileges.
- Users can also be granted privileges directly, but using roles is a better practice for managing permissions.

Roles Can Be Granted Privileges

- Roles can be granted privileges on database objects (tables, schemas, databases).

```
1  -- Database-level privilege
2  GRANT CONNECT ON DATABASE company_db TO sales_dept;
3
4  -- Schema-level privilege
5  GRANT USAGE ON SCHEMA sales TO sales_dept;
6
7  -- Table-level privilege
8  GRANT SELECT ON sales.customers TO sales_dept;
```


Users Can Be Granted Roles

- Users can be granted roles to inherit their privileges.

```
1 GRANT sales_dept TO alice;  
2 GRANT finance_dept TO bob;
```

Users Can Also Be Granted Privileges Directly

- 1 **GRANT SELECT ON** sales.customers **TO** alice;
- 2 **GRANT SELECT ON** finance.invoices **TO** bob;

Granting Several Privileges at Once

```
1 GRANT SELECT, INSERT, UPDATE ON sales.customers TO sales_dept;
```

ALL Privilege Keyword

- The ALL keyword is a shorthand to grant or revoke all available privileges on a database object.

```
1  -- Table level privilege
2  GRANT ALL ON sales.customers TO sales_dept;
3
4  -- Schema level privilege
5  GRANT ALL ON SCHEMA sales TO sales_dept;
6
7  -- Database level privilege
8  GRANT ALL ON DATABASE company_db TO sales_dept;
```

System Level Privileges

- System-level privileges control access to the entire PostgreSQL instance.
- These include permissions like CREATE ROLE, CREATE DATABASE, and SUPERUSER.
- SUPERUSER: Grants all privileges, including bypassing all permission checks (use with caution).

```
1 -- Create a superuser role
2 CREATE ROLE db_admin CREATEDB CREATEROLE LOGIN PASSWORD 'secret';
```

System-Level Privileges

- **CREATEDB:** Allows a role to create new databases.
- **CREATEROLE:** Allows a role to create, modify, or drop other roles.
- **SUPERUSER:** Grants unrestricted privileges on the entire database cluster, bypassing all access restrictions.

System-Level Privileges

- **REPLICATION**: Grants a role the ability to initiate streaming replication and create replication slots.
- **BYPASSRLS**: Grants a role the ability to bypass Row-Level Security policies.
- **LOGIN**: Allows the role to log in and connect to a database (this is a prerequisite for checking the CONNECT privilege).

Database-Level Privileges

- Database-level privileges in PostgreSQL are permissions that control what actions a role (user or group) can perform on a specific database instance.
- The `GRANT ... ON DATABASE` command is used to assign special permissions that apply to the database as a whole.

Database-Level Privileges

- **CONNECT**: This privilege allows a role to connect to the specified database. Without this privilege, a role cannot access the database at all.
- **CREATE**: This privilege allows a role to create new schemas within the specified database.

```
1 -- Allow connection to database
2 GRANT CONNECT ON DATABASE company_db TO sales_dept;
3
4 -- Allow creating schemas
5 GRANT CREATE ON DATABASE company_db TO finance_dept;
```

Schema-Level Privileges

- Schema-level privileges control access to specific schemas within a database.
- The GRANT ... ON SCHEMA command is used to assign permissions that apply to all objects within a specific schema.
- The Two Core Schema-Level Privileges:
 - **USAGE**: This privilege allows a role to access objects within the specified schema.
 - **CREATE**: This privilege allows a role to create new objects (tables, views, functions) within the specified schema.

USAGE — The “Visibility” Privilege

- The fundamental permission to access or see objects within a schema.
- It's the key to the front door of a building (the schema). Without it, you can't get inside to see the rooms (the tables), even if you have keys to those individual rooms.
- A user cannot access any object in a schema without the USAGE privilege on that schema.

CREATE — The “Builder” Privilege

- The permission to create new objects (like tables, functions, etc.) within a schema.
- It's the permission to build new furniture inside the building.
- A user must also have the USAGE privilege to be able to CREATE. He can only build in a schema if he can see it.

Table-Level Privileges

- Table-level privileges control access to specific tables within a schema.
- Common privileges include SELECT, INSERT, UPDATE, DELETE, and REFERENCES.

Table-Level Privileges

```
1  -- Grant different levels of access
2  GRANT SELECT ON sales.customers TO sales_dept;
3  GRANT INSERT, UPDATE ON sales.customers TO sales_dept;
4  GRANT SELECT ON sales.customers TO finance_dept; -- Read-only
5
6  GRANT SELECT, INSERT, UPDATE ON finance.invoices TO finance_dept;
```

Scenario 1: The Read-Only Analyst

You want a user to be able to query tables but not create anything.

```
1 -- Allow the user to "enter" and see the contents of the schema sales
2 GRANT USAGE ON SCHEMA sales TO analyst_user;
3
4 -- Note: You still need to grant SELECT on the specific tables inside.
5 GRANT SELECT ON TABLE sales.customers TO analyst_user;
```

Scenario 2: The Developer

You want a developer to be able to create and manage tables in a dedicated schema.

```
1 -- Grant both USAGE and CREATE in a single command
2 GRANT USAGE, CREATE ON SCHEMA analytics TO developer_user;
```


Column-Level Security

- Column-level security allows restricting access to specific columns within a table.
- This is useful for sensitive data that should not be exposed to all users.

Imagine a table with the following columns:

employee_id	name	department	start_date	salary	social_security_number
101	Alice	Sales	2023-05-10	95000	XXX-XX-XXXX
102	Bob	IT	2022-11-20	110000	XXX-XX-XXXX

- **Public Data:** `employee_id`, `name`, `department`
- **Sensitive Data:** `salary`, `social_security_number`

The goal is to allow a general user (like an analyst) to see the public data without ever being able to access the sensitive columns.

Example of Direct Column GRANTS

PostgreSQL allows you to grant permissions (**SELECT**, **UPDATE**, etc.) on a column-by-column basis using the **GRANT** command.

```
1 -- First, make sure the role has no access to the full table
2 REVOKE SELECT ON employees FROM analyst;
3
4 -- Then, grant access only to the specific columns they can see
5 GRANT SELECT (employee_id, name, department, start_date)
6 ON employees TO analyst;
```

Views as a Security Layer (Abstraction approach)

Instead of giving users direct access to the table, you create a **view**. A view is a virtual table based on a query, which acts as a secure “lens” into the real data.

How It Works

1. **Create a view** that only selects the non-sensitive columns.
2. **Deny access** to the underlying (base) table.
3. **Grant access** to the new view instead.

Example of Views for Column-Level Security

```
1  -- 1. Create a "safe" view for analysts
2  CREATE VIEW employee_public_data AS
3      SELECT employee_id, name, department, start_date
4      FROM employees;
5
6  -- 2. Revoke access to the real table
7  REVOKE ALL ON employees FROM analyst;
8
9  -- 3. Grant access to the view
10 GRANT SELECT ON employee_public_data TO analyst;
```

Comparing the Two Approaches

Feature	Direct GRANTS	Views as a Security Layer
Basic Concept	Permissions are applied directly to the columns of the base table.	A virtual table (view) is created that exposes only the safe columns. Users are given permission on the view, not the base table.
User Experience	Users must explicitly name the columns they have access to. A SELECT * query will fail with a permission error.	Transparent to the user. They can query the view as if it were a normal table, and SELECT * works seamlessly on the view.
Management	Can become complex. Each time a new role needs similar access, permissions must be granted column by column. Harder to audit.	Simple and centralized. You define the security logic once in the view. To change access for all users of that view, you only alter the view.
Flexibility	Limited to just column permissions.	Highly flexible. The view's WHERE clause can also enforce

Feature	Direct GRANTS	Views as a Security Layer
		row-level security , and columns can be renamed or computed.
Security	Secure, but exposes the underlying table structure to the user, even if they can't access all of its data.	More robust abstraction. It completely hides the existence of the sensitive columns and the underlying table from the end-user.
Best Use Case	Good for simple, one-off rules or when you have very few roles and tables to manage.	The standard and recommended approach for most scenarios, especially in complex applications with multiple user roles.

Row-Level Security (RLS)

- Row-Level Security (RLS) is a powerful PostgreSQL feature that controls which rows a user can access within a table.
- Instead of granting access to an entire table, RLS applies an automatic, invisible WHERE clause to every query, filtering the data based on the user running the query. From the user's perspective, the rows they cannot see simply do not exist.

Why Use RLS?

- RLS is the foundation for building secure, multi-user applications directly into the database.
- **User-Specific Data:** Allow users to see and manage only their own records (e.g., their own tasks, orders, or profile information).
- **Multi-Tenant SaaS:** Ensure a customer from Company A can never see data belonging to Company B.
- **Hierarchical Access:** A manager can see the records of their direct reports, but not the records of other managers.

Enable RLS on a Table

- Row-Level Security must be enabled on a table
- Policies should be created
- A policy defines the conditions under which rows are visible to users.
- Policies can be based on user roles, session variables, or other criteria.

A Practical Example of RLS

Let's consider the following scenario: a **tasks** table where users can only see their own tasks.

```
1 CREATE TABLE tasks (  
2   task_id SERIAL PRIMARY KEY,  
3   owner_username TEXT NOT NULL,  
4   task_description TEXT  
5 );
```

A Practical Example of RLS

Step 1: Enable RLS on the Table

RLS policies have no effect until you explicitly enable them on the table. This is the master “on” switch.

```
1 ALTER TABLE tasks ENABLE ROW LEVEL SECURITY;
```

A Practical Example of RLS

Step 2: Create the Security Policy

This policy defines the rule: a row is only visible if the `owner_username` matches the person running the query. PostgreSQL's `current_user` function provides the username of the active session.

```
1 CREATE POLICY user_can_access_own_tasks
2 ON tasks
3 FOR ALL -- Applies to SELECT, INSERT, UPDATE, DELETE
4 USING (owner_username = current_user);
```

The Result

- When user `alice` runs `SELECT * FROM tasks;`, she only sees rows where `owner_username = 'alice'`.
- When user `bob` runs the same query, he only sees rows where `owner_username = 'bob'`.

Enable RLS on a Table

- This is done using the **ALTER TABLE** command.
- Once RLS is enabled, access to the table is restricted based on defined policies.

```
1 -- Enable RLS
2 ALTER TABLE sales.customers ENABLE ROW LEVEL SECURITY;
```

Create RLS Policies

- Policies define who can see which rows.
- The syntax is as follows:

```
1 CREATE POLICY policy_name ON table_name
2   FOR { ALL | SELECT | INSERT | UPDATE | DELETE }
3   TO { role_name | PUBLIC }
4   USING ( condition );
```

Example of RLS Policies

- In the following example, we create a policy for the sales representatives to see only their own customers.
- sales_dept is a role assigned to sales reps
- The condition checks that the sales_rep_id matches the current user's employee_id
- current_user is a built-in function that returns the name of the current database user.
- sales_rep_id is a column in the sales.customers table that identifies the sales representative responsible for the customer.

```
1 -- Create policy for sales reps (see only their customers)
2 CREATE POLICY sales_rep_policy ON sales.customers
3     FOR ALL
4     TO sales_dept
5     USING (sales_rep_id = (
6         SELECT employee_id
7         FROM hr.employees
```

```
8         WHERE email = current_user || '@company.com'  
9     ));
```


Example of RLS Policies

- For managers, we create a policy that allows them to see all customers without restrictions.
- sales_manager is a role assigned to managers.
- The condition is simply true, meaning no restrictions apply.

```
1  -- Managers can see all customers
2  CREATE POLICY manager_policy ON sales.customers
3      FOR ALL
4      TO sales_manager
5      USING (true);  -- No restrictions
```

Default Privileges

- Default privileges allow you to set permissions that will automatically apply to future objects created by a specific role in a specific schema.
- This simplifies permission management by ensuring that new objects inherit the desired access controls.

Set up automatic privilege assignment

```
1  -- Future tables created by finance_dept
2  ALTER DEFAULT PRIVILEGES
3      FOR ROLE finance_dept
4      IN SCHEMA finance
5      GRANT SELECT ON TABLES TO sales_dept;
6
7  -- Revoke public access by default
8  ALTER DEFAULT PRIVILEGES
9      REVOKE ALL ON TABLES FROM PUBLIC;
```

Real-World Security Setup

E-commerce application security

```
1 -- 1. Create application roles
2 CREATE ROLE ecommerce_app;
3 CREATE ROLE customer_service;
4 CREATE ROLE analytics_team;
5 CREATE ROLE admin_team;
6
7 -- 2. Create schemas
8 CREATE SCHEMA products;
9 CREATE SCHEMA orders;
10 CREATE SCHEMA customers;
11 CREATE SCHEMA analytics;
```

Real-World Security Setup

```
1 -- 3. Set up basic privileges
2 GRANT USAGE ON SCHEMA products TO ecommerce_app, customer_service, analytics_team;
3 GRANT USAGE ON SCHEMA orders TO ecommerce_app, customer_service, analytics_team;
4 GRANT USAGE ON SCHEMA customers TO ecommerce_app, customer_service;
5 GRANT USAGE ON SCHEMA analytics TO analytics_team;
6
7 -- 4. Application gets read/write on operational data
8 GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA products TO ecommerce_app;
9 GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA orders TO ecommerce_app;
10 GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA customers TO ecommerce_app;
```

Real-World Security Setup

```
1  -- 5. Customer service gets limited access
2  GRANT SELECT ON ALL TABLES IN SCHEMA products TO customer_service;
3  GRANT SELECT, UPDATE ON ALL TABLES IN SCHEMA orders TO customer_service;
4  GRANT SELECT ON ALL TABLES IN SCHEMA customers TO customer_service;
5
6  -- 6. Analytics gets read-only access
7  GRANT SELECT ON ALL TABLES IN SCHEMA products TO analytics_team;
8  GRANT SELECT ON ALL TABLES IN SCHEMA orders TO analytics_team;
9  GRANT SELECT ON ALL TABLES IN SCHEMA customers TO analytics_team;
10 GRANT ALL ON SCHEMA analytics TO analytics_team;
```

Advanced RLS Example

```
1 -- Create tenant-aware table
2 CREATE TABLE orders.customer_orders (
3     order_id SERIAL PRIMARY KEY,
4     customer_id INTEGER,
5     tenant_id INTEGER, -- Multi-tenant key
6     order_date DATE,
7     total DECIMAL(10,2)
8 );
```

Multi-tenant Application

```
1  -- Create tenant-aware table
2  CREATE TABLE orders.customer_orders (
3      order_id SERIAL PRIMARY KEY,
4      customer_id INTEGER,
5      tenant_id INTEGER, -- Multi-tenant key
6      order_date DATE,
7      total DECIMAL(10,2)
8  );
9
10 -- Enable RLS
11 ALTER TABLE orders.customer_orders ENABLE ROW LEVEL SECURITY;
12
13 -- Policy for tenant isolation
14 CREATE POLICY tenant_isolation ON orders.customer_orders
15     FOR ALL
16     TO ecommerce_app
17     USING (tenant_id = current_setting('app.current_tenant_id')::INTEGER);
18
19 -- Policy for customer service (specific tenant)
```

Multi-tenant Application

Using the tenant system

```
1 -- Set tenant context
2 SET app.current_tenant_id = '123';
3
4 -- Now queries only see tenant 123's data
5 SELECT * FROM orders.customer_orders;
```


Security Functions and Views

Useful security queries

```
1  -- Check current privileges
2  SELECT
3      table_schema,
4      table_name,
5      privilege_type,
6      grantee
7  FROM information_schema.table_privileges
8  WHERE grantee = 'sales_dept';
9
10 -- See role memberships
11 SELECT
12     r.rolname AS role_name,
13     m.rolname AS member_name
14 FROM pg_roles r
15 JOIN pg_auth_members am ON r.oid = am.roleid
16 JOIN pg_roles m ON am.member = m.oid
17 ORDER BY r.rolname, m.rolname;
18
19 -- Check RLS policies
```

Using psql for User Management

- psql provides interactive commands to manage users and roles directly from the command line.
- This is useful for quick checks and modifications without writing full SQL scripts.
- You can also use it to test queries and explore the database schema.

Interactive user management

```
1 # Connect as superuser
2 psql -U postgres
3
4 # Create user interactively
5 \password new_user_name
6
7 # List all users and roles
8 \du
9
10 # Check database permissions
11 \l
12
13 # Connect as different user
```

```
14 \c database_name user_name
15
16 # Check current connection info
17 \conninfo
18
19 # Show current user
```

Password Security

Secure password practices

```
1 -- Create user with strong password policy
2 CREATE USER secure_user WITH
3     PASSWORD 'ComplexP@ssw0rd2024!'
4     VALID UNTIL '2025-12-31';
5
6 -- Force password change on next login
7 ALTER USER secure_user PASSWORD 'temp_password' VALID UNTIL 'tomorrow';
8
9 -- Remove password (disable password auth)
10 ALTER USER api_service PASSWORD NULL;
```

Password encryption

```
1 -- Check password encryption
2 SHOW password_encryption;
3
4 -- Set to stronger encryption
5 ALTER SYSTEM SET password_encryption = 'scram-sha-256';
6 SELECT pg_reload_conf();
```

Security Best Practices

- ✓ **Principle of least privilege** - Give minimum necessary access
- ✓ **Use roles for grouping** - Don't grant privileges to individual users
- ✓ **Enable SSL/TLS** - Encrypt data in transit
- ✓ **Regular password rotation** - Change passwords periodically
- ✓ **Audit access regularly** - Review who has access to what
- ✓ **Use RLS for sensitive data** - Fine-grained access control

Common Security Mistakes

- ❌ **Using superuser for applications** - Too much privilege
- ❌ **Granting to PUBLIC** - Everyone gets access
- ❌ **Weak passwords** - Easy to guess or crack
- ❌ **No SSL in production** - Data travels in plaintext
- ❌ **Ignoring failed login attempts** - No monitoring
- ❌ **Shared accounts** - Can't track individual access

Troubleshooting Access Issues

Common permission problems

```
1  -- Check if user can connect to database
2  SELECT has_database_privilege('username', 'database_name', 'CONNECT');
3
4  -- Check table privileges
5  SELECT has_table_privilege('username', 'schema.table_name', 'SELECT');
6
7  -- Check column privileges
8  SELECT has_column_privilege('username', 'schema.table_name', 'column_name', 'SELECT');
9
10 -- See effective privileges
11 \dp schema.table_name
```

Real-World Security Scenario

Banking application security

```
1  -- Create role hierarchy
2  CREATE ROLE bank_employees;
3  CREATE ROLE tellers;
4  CREATE ROLE loan_officers;
5  CREATE ROLE managers;
6  CREATE ROLE auditors;
7
8  -- Set up inheritance
9  GRANT bank_employees TO tellers;
10 GRANT bank_employees TO loan_officers;
11 GRANT tellers TO managers;
12 GRANT loan_officers TO managers;
13
14 -- Create schemas
15 CREATE SCHEMA accounts;
16 CREATE SCHEMA transactions;
17 CREATE SCHEMA loans;
18 CREATE SCHEMA audit;
19
```


Summary

PostgreSQL provides comprehensive security at multiple levels

Security layers:

- Authentication (who you are)
- Authorization (what you can do)
- Access control (where you can go)
- Row-level security (what data you see)

Remember: - Start with least privilege - Use roles for organization - Monitor and audit regularly - Keep security configurations simple and maintainable - Security is a continuous process: review and update as needed